

Detailed Project Report: Financial Accounting Prototype

This document provides a comprehensive technical overview of the "Printos" Financial Accounting Prototype. It is designed for developers building the backend and frontend systems, detailing every module, calculation logic, data structure, and UI behavior present in the prototype.

1. System Architecture & State Management

The prototype currently uses local storage to persist data. The backend should replace this with a relational database (e.g., PostgreSQL, MySQL).

Core Entities / Data Stores:

- transactions: Array of all transactions (Sales, Purchases, Incomes, Expenses, Assets, Capital).
- stockList: Array of inventory items.
- partyList: Array of Debtors (Customers) and Creditors (Suppliers).
- quotationList: Array of quotes generated.
- expenseList: Array of manual expense entries.
- ledgerList: Array of double-entry ledger records (DR/CR).
- companyData: Object containing company profile, settings (e.g., netting), logo, and QR code.

2. Calculation Logic by Module

2.1 Sales Module

The Sales module allows creating invoices, calculating taxes, discounts, and tracking payment status.

Row-level Calculations:

- **Gross Amount:** $\text{Gross} = \text{Quantity} * \text{Price}$
- **Discount:** Can be entered as a percentage (discPerc) or absolute amount (discAmt).
 - If Percentage entered: $\text{discAmt} = \text{Gross} * (\text{discPerc} / 100)$
 - If Amount entered: $\text{discPerc} = (\text{discAmt} / \text{Gross}) * 100$
- **Taxable Amount:** $\text{Taxable} = \text{Gross} - \text{discAmt}$ (minimum 0).
- **Tax Calculation (Depends on Global Price Type):**
 - If "Without Tax" (Exclusive): $\text{Tax Amount} = \text{Taxable} * (\text{taxPerc} / 100)$
 - If "With Tax" (Inclusive):
 - $\text{Base} = \text{Taxable} / (1 + (\text{taxPerc} / 100))$
 - $\text{Tax Amount} = \text{Taxable} - \text{Base}$
 - $\text{Taxable} = \text{Base}$
- **Row Total:** $\text{Row Total} = \text{Taxable} + \text{Tax Amount}$

Invoice-level Calculations:

- **Grand Total:** Sum of all Row Total values.

- **Round-off:** If the round-off checkbox is checked, $\text{Grand Total} = \text{Math.round}(\text{Grand Total})$.
- **Balance Calculation:**
 - $\text{Balance} = \text{Grand Total} - \text{Paid Amount}$ (minimum 0).
 - *Note:* If the Payment Mode is "Cash", Paid Amount is forced to equal Grand Total (Balance = 0).
- **Stock Deduction:** Upon saving, Quantity sold is subtracted from the respective item in stockList.

2.2 Purchase Module

The Purchase module tracks inward inventory and supplier payables.

Row-level Calculations:

- **Taxable Amount:** $\text{Taxable} = (\text{Quantity} * \text{Price}) - \text{Discount Amount}$
- **Tax Amount:** $\text{Tax Amount} = \text{Taxable} * (\text{taxPerc} / 100)$
- **Row Total:** $\text{Row Total} = \text{Taxable} + \text{Tax Amount}$

Voucher-level Calculations:

- **Total Taxable:** Sum of all Taxable values.
- **Total Tax:** Sum of all Tax Amount values.
- **Grand Total:** $\text{Total Taxable} + \text{Total Tax}$. Rounded mathematically if round-off is enabled.
- **Balance Calculation:** Similar to Sales. $\text{Balance} = \text{Grand Total} - \text{Paid Amount}$.
- **Stock Addition:** Upon saving, Quantity purchased is added to the respective item in stockList.

2.3 Parties (Debtors & Creditors) Module

Manages balances to receive (Debtors) and to pay (Creditors).

- **Types:** Receive (Asset/Debtor) or Pay (Liability/Creditor).
- **Balance Tracking:**
 - The prototype dynamically updates party.balance and party.type when sales, purchases, or settlements occur.
 - *Credit Sale:* Increases Receive balance.
 - *Credit Purchase:* Increases Pay balance.
 - *Settlement/Payment:* Reduces the respective balance. If a payment exceeds the current balance, the party's type flips (e.g., from Receive to Pay).

2.4 Outstanding & Settlements

Tracks pending invoices and bills.

- **Identification:** Filters transactions where $\text{balance_amount} > 0$.
- **Overdue Logic:** An invoice is overdue if $\text{Current Date} > \text{due_date}$. Calculates days overdue as $(\text{Current Date} - \text{due_date})$ in days.
- **Settlement Logic:**
 - Reduces balance_amount of the specific transaction.
 - Updates paid_amount.
 - Updates partyList running balance.
 - Creates necessary ledgerList entries for the receipt/payment.

2.5 Financial Statements

Generates real-time Trading Account, Profit & Loss Account, and Balance Sheet.

A. Trading Account:

- Purchases: Sum of all transactions where type === 'Purchase'.
- Sales: Sum of all transactions where type === 'Sale'.
- Closing Stock: Sum of (Qty * Price) for all items in stockList.
- Gross Profit: (Sales + Closing Stock) - Purchases. (If negative, it is a Gross Loss).

B. Profit & Loss Account:

- Expenses: Sum of transactions where type === 'Expense' plus all entries in expenseList.
- Incomes: Sum of transactions where type === 'Income'.
- Net Profit: (Gross Profit + Incomes) - Expenses. (If negative, it is a Net Loss).

C. Balance Sheet:

- **Accounting Preference (Netting vs Gross):** Based on companyData.netting_enabled.
 - *Netting Enabled:* Simply takes the current running balance of each party.
 - *Gross Logic:* Recalculates from scratch using initial_balance + dynamically summing all related transactions (Sales, Purchases, Receipts, Payments) to get exact Gross Debtors and Gross Creditors.
- Cash Balance: Sum of all Cash receipts minus Cash payments.
- Bank Balance: Sum of all Bank receipts minus Bank payments.
- Total Assets: Assets (Transactions) + Closing Stock + Cash Balance + Bank Balance + Gross Debtors.
- Total Liabilities: Capital (Transactions) + Net Profit + Gross Creditors.
- *Validation:* The system checks if Total Assets == Total Liabilities. If they do not match, it displays a Mismatch warning.

2.6 Ledger Module (Double Entry System)

Maintains strict debit/credit (DR/CR) rules.

- Automatically creates entries for every financial action.
 - *Example - Credit Sale:* DR Customer Account, CR Sales Account.
 - *Example - Expense Payment:* DR Expense Account, CR Cash/Bank Account.
- **Running Balance Calculation:**
 - If the account is an Asset/Debtor: Balance = Opening Balance + DR - CR. (If Balance < 0, it displays as CR).
 - If the account is a Liability/Creditor: Balance = Opening Balance + CR - DR. (If Balance < 0, it displays as DR).

2.7 Stock Module

- **Valuation:** Follows a simple Total Value = Qty * Price formula.
- **Validation:** When creating a Sale, the system checks if the sold quantity exceeds available stock. It prevents negative stock.

3. Developer Guidelines

- **Precision:** All monetary calculations should be done using high-precision decimal types on the backend (e.g., DECIMAL(10,2) in SQL, or similar) to avoid floating-point inaccuracies.
- **Concurrency:** Implement database transactions when saving Sales or Purchases. Deducting stock, updating party balances, and creating ledger entries must happen atomically.
- **Taxation (India Context):**
 - The UI splits taxes into CGST/SGST if the supply is intra-state (Company State == Customer State).
 - Uses IGST if the supply is inter-state (Company State != Customer State).
 - The backend should store the total tax percentage and calculate the splits dynamically or store them explicitly.

4. UI/UX Behaviors (To replicate in Frontend)

- Double-clicking rows in tables (Parties, Stock, Transactions) opens the edit modal.
- Live invoice preview updates in real-time as users type or change rows.
- Badges dynamically show stock availability ("In Stock" vs "Low Stock").
- Deleting a transaction triggers a complete reversal (Stock is added back, Party balances are restored, Ledger entries are deleted).

End of Document